

CD-CureNO: Reproducible software for restriction-preserving causal neural operators in composite curing

Hyojin Park^a, Nam-Hyun Yoo^b, Jinhong Yang^{c,*}

^a*Gyeongnam Intelligence Innovation Center (GIIC), Kyungnam University, 7
Gyeongnamdaehak-ro, Masanhappo-gu, Changwon 51767, Republic of Korea*

^b*Computer Engineering, Kyungnam University, Changwon 51767, Republic of Korea*

^c*Department of Medical Information Technology, Inje University, Gimhae 50834,
Republic of Korea*

Abstract

CD-CureNO is a Python research-software stack for reproducible operator learning in thermochemical composite curing. It audits legacy position-wise ResFNO workflows, implements conservative one- and two-dimensional solvers, creates immutable datasets and splits, trains joint and causal operators, inflates one-dimensional checkpoints into restriction-preserving two-dimensional models, and independently verifies saved experiments. Scientific safeguards include train-only normalization, complete-case splits, held-out firewalls, transactional run journals, hash-bound manifests, causal future-perturbation tests, and conservative energy diagnostics. In staged validation, a 512-case two-dimensional benchmark passed 27 solver checks and deterministic replay, while checkpoint inflation preserved homogeneous solutions to numerical tolerance. The package supports auditable development without converting validation-only evidence into confirmatory claims.

Keywords: research software, neural operator, composite curing, thermochemical simulation, reproducibility, causal machine learning

Table 1: SoftwareX code metadata. The C1–C8 labels and field meanings follow the official SoftwareX original-software template, version 6 (March 2026).

Nr	Code metadata description	Metadata
C1	Current code version	v0.0.1
C2	Permanent link to code/repository used for this code version	https://github.com/Jinhong-Yang/CD-CureN0/tree/v0.0.1
C3	Legal code license	Apache License 2.0 (SPDX: Apache-2.0)
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python 3.10–3.12; PyTorch 2.13.0; NumPy 2.2.6; SciPy 1.15.3; pandas 2.3.3; PyArrow 25.0.0; Matplotlib 3.10.9; pytest 9.1.1
C6	Compilation requirements, operating environments, and dependencies	Python package installed with <code>pip install -e ".[dev]"</code> ; CPU execution is supported for audits and tests; CUDA is optional for model training; dependencies are pinned in <code>pyproject.toml</code> .
C7	Link to developer documentation/manual	https://github.com/Jinhong-Yang/CD-CureN0
C8	Support email for questions	jinhong@inje.ac.kr

1 Code metadata

1. Motivation and significance

Thermochemical curing models connect an imposed air-temperature program to the temperature and degree-of-cure fields inside a composite-tool assembly. These fields matter because local thermal overshoot, cure gradients, and incomplete cure can affect process design and final part quality. High-fidelity finite element or finite-volume calculations are useful but costly when cure cycles, heat-transfer coefficients, material parameters, and geometry must be varied. Neural operators offer a function-to-function surrogate rather than a model for one fixed input, and Fourier neural operators provide one influential implementation of that idea [1].

*Corresponding author

Email address: jinhong@inje.ac.kr (Jinhong Yang)

Existing composite-curing work already includes physics-informed neural networks [2], residual neural operators [3], and physics-informed neural operators [4]. CD-CureNO therefore does not claim novelty for using an FNO, jointly predicting temperature and cure, or adding a physics loss. Its software problem is narrower and more practical: how can a research team develop, transfer, evaluate, and report cross-dimensional curing operators without losing the provenance and scientific controls needed to interpret the result?

The motivating legacy ResFNO implementation learns one temporal operator at a fixed through-thickness position and reconstructs a field by stacking up to 51 independently trained models [5, 3]. A public implementation is scientifically valuable, but direct reuse exposes common research-code risks: normalization before splitting, inconsistent code paths, batch-local metric aggregation, incomplete field outputs, and ambiguity about whether a displayed position-by-time image is a jointly coupled spatial operator. CD-CureNO provides a behaviorally faithful legacy path and a separately named corrected path, then moves to a single joint 1+1-D model and a true two-spatial-dimensional target. The terminology is enforced in code, manifests, and reporting artifacts.

Lower-dimensional pretraining and boundary-to-domain lifting have become important precedents for data-efficient operator learning [6, 7]. CD-CureNO adds software support for a stronger engineering invariant: for laterally homogeneous coefficients, initial conditions, and boundary data with zero lateral flux, the target model should recover the source mapping when initialized. The implementation copies the shared through-thickness and temporal computation, initializes new lateral branches as zero residuals, and independently recreates every target tensor before accepting an inflated checkpoint. A separate causal path uses dilated temporal convolutions and a future-perturbation test, following the broader principle that causal response models must exclude future inputs [8].

The intended users are researchers in scientific machine learning, composite manufacturing, and computational heat transfer who need more than model classes. They need immutable split manifests, conservative reference solvers, resumable runs, provenance-rich outputs, independent postflight checks, and a claim boundary that distinguishes validation-only evidence from confirmatory held-out results. CD-CureNO packages those concerns as one inspectable workflow.

2. Software description

2.1. Software architecture

CD-CureNO follows a staged artifact flow rather than a notebook-only workflow. Figure 1 summarizes the five principal layers. Data and provenance enter through immutable manifests. Conservative solvers generate or validate labels. Model modules implement joint, causal, and dimension-transfer operators. Training modules provide resume support and single-writer journals. Evaluation and reporting modules independently reconstruct metrics, bind numbers to run identifiers, and fail closed when an integrity contract is not satisfied.

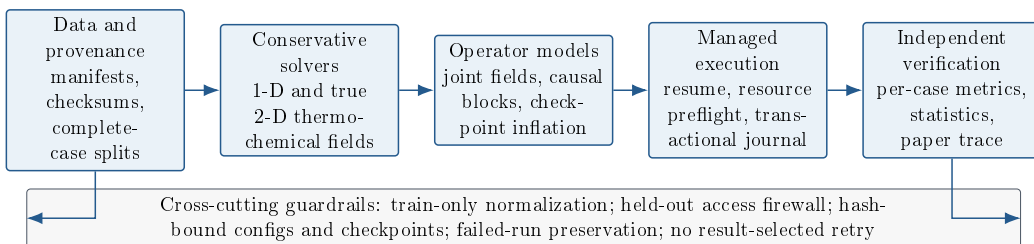


Figure 1: CD-CureNO software architecture. Each stage emits inspectable artifacts consumed by the next stage; cross-cutting guardrails are verified again during independent aggregation and reporting.

The package is organized under `src/cdcureno`. The `data` module loads public and generated arrays, fits normalization on training cases only, and exposes separate training shards and evaluation readers. The `solvers` module implements conservative one- and two-dimensional thermochemical integration with harmonic face conductance and Robin boundary fluxes. The `models` module contains joint operators, causal temporal blocks, target operators, ablations, and deterministic checkpoint-inflation logic. The `training` module records configurations, environment and Git state, histories, checkpoints, receipts, and process liveness. The `evaluation` and `reporting` modules recompute complete-case metrics from saved predictions and bind manuscript values to exact evidence.

2.2. Restriction-preserving and causal target construction

The target model uses canonical input order $[B, N_t, N_z, N_x, C_{\text{in}}]$ and returns temperature and degree of cure over the same time-space grid. For a

Table 2: Major software functionality and its primary audit artifact.

Function	User-facing capability	Audit output
Legacy audit	Inspect upstream data shapes, splits, normalization, losses, and expected files without modifying the pinned source	JSON/Markdown inventory and regression tests
Reproducible training	Run exact, corrected, joint, and causal experiments with deterministic IDs and resume support	Resolved config, Git/environment snapshot, history, checkpoints, predictions, receipt
Conservative simulation	Generate one- and two-dimensional temperature/cure labels and validate energy closure, refinement, and extrusion	Per-case metadata, convergence tables, immutable manifests
Cross-dimensional transfer	Inflate a one-dimensional source into a target with copied shared tensors and zero-residual lateral adapters	Tensor mapping, tensor hashes, recreated target state, restriction metrics
Causality verification	Perturb all future input channels and compare every saved output on the unperturbed prefix	Cutoff-wise prefix differences and pass/fail contract
Independent reporting	Recompute per-case metrics, release frozen populations, run registered statistics, and trace manuscript values	Postflight reports, release receipts, statistics bundle, number-to-run trace

73 laterally extruded source prediction, the target is written conceptually as

$$\widehat{T}_{2D}(x, z, t) = \text{Lift}_x \left[\widehat{T}_{1D}(z, t) \right] + \Delta T_x(x, z, t). \quad (1)$$

74 The new geometry projection and lateral adapter output factors are initial-
75 ized to zero. Consequently, homogeneous inputs recover the source compu-
76 tation at initialization, up to floating-point ordering effects for $N_x > 1$. The
77 inflation report records copied tensors, initialized tensors, shape changes,

78 and hashes. A verifier then reconstructs the full untrained target from the
79 pinned source checkpoint, configuration, and seed; checking model outputs
80 alone is insufficient because a trained target could accidentally appear lateral-
81 invariant on a small test.

82 The structurally causal target contains eight dilated temporal convolu-
83 tions and no temporal FFT modules. Causality is checked by perturbing ev-
84 ery input channel after several cutoffs and requiring all temperature, residual,
85 degree-of-cure, and cure-rate prefixes to remain unchanged. Cure is repre-
86 sented through a nonnegative latent rate integrated into a bounded monotone
87 state, while conservative residuals use finite-volume-compatible fluxes rather
88 than global derivatives across a material interface.

89 *2.3. Scientific and execution safeguards*

90 Every expensive run has a resolved configuration, data checksums, ran-
91 dom seed, environment snapshot, checkpoint history, per-case metrics, and
92 physical-unit predictions. Splits are defined by complete cure cycle or com-
93 plete simulation case; spatial and temporal grid points are never treated as
94 independent samples. The package preserves failed and superseded runs, and
95 result indices record only independently verified completions. Managed ex-
96 ecution uses a single-writer lease and journal so interruption cannot silently
97 change the selected checkpoint population.

98 The reporting layer separates structural, validation-only, descriptive, and
99 confirmatory evidence. The current SoftwareX article uses only passed P0–
100 P5 software-validation artifacts. It does not use the unfinished P6 held-out
101 campaign to claim superiority, label savings, state of the art, or broad source-
102 seed invariance.

103 **3. Illustrative examples**

104 *3.1. Installation and audit*

105 The repository targets Python 3.10–3.12. A clean editable installation
106 and CPU test run are:

```
107 python -m venv .venv  
108 .venv/Scripts/python -m pip install -e ".[dev]"  
109 .venv/Scripts/python -m pytest -q  
110
```

112 The legacy audit is read-only and supports a dry run:

```

113
114 .venv/Scripts/python scripts/audit_legacy_repo.py \
115     --repo-root external/ResFNO \
116     --output-dir outputs/audit \
117     --dry-run
118

```

119 The implemented CLI also exposes auditable exact, corrected, joint fac-
120 torized, and joint causal training modes. Independent evaluation reloads a
121 saved run by identifier and recomputes its complete-case metrics.

122 3.2. Conservative two-dimensional benchmark

123 The two-dimensional workflow separates plan generation from label gen-
124 eration, which allows users to review case families and split semantics before
125 solving:

```

126
127 .venv/Scripts/python scripts/generate_p4_2d.py \
128     --tier core --plan-only
129 .venv/Scripts/python scripts/validate_p4_solver.py
130 .venv/Scripts/python scripts/verify_p4_gate.py
131

```

132 The frozen core benchmark contains 512 cases with temperature/cure
133 field shape [512, 112, 50, 40]. The production solver passed 27 pre-specified
134 checks; all cases completed, the maximum relative global energy residual
135 was 1.345×10^{-12} , and 11 cases selected from plan metadata reproduced
136 their saved float32 slice hashes exactly. The independent SciPy-BDF check
137 reassembled fluxes and time integration but shared the grid, material values,
138 and cure law, so it is not described as a fully independent external solver.

139 3.3. Checkpoint inflation and restriction verification

140 The causal inflation path can execute all checks in memory before writing:

```

141
142 .venv/Scripts/python \
143     scripts/inflate_causal_1d_to_2d_checkpoint.py --dry-run
144 .venv/Scripts/python \
145     scripts/verify_p5_causal_restriction.py --dry-run
146

```

147 For the saved causal target, 100 source tensors were copied bitwise and
148 17 target tensors were initialized; the complete 117-tensor state matched a
149 deterministic reconstruction. On the actual homogeneous P4 validation in-
150 put, temperature restriction relative L_2 and lateral-invariance scores were
151 3.33×10^{-8} and 1.76×10^{-8} , respectively. The maximum future-prefix dif-
152 ference was exactly zero across three cutoffs, four output fields, and pertur-
153 bations to all 20 target channels. These checks use label-free homogeneous

154 inputs and establish initialization behavior and architectural causality, not
 155 predictive superiority.

156 3.4. Joint-field reconstruction

157 Figure 2 illustrates an output of the earlier joint 1+1-D stage. The case
 158 identifier was fixed before plotting, and shared color scales permit visual
 159 comparison. The figure is deliberately labelled as a through-thickness-by-
 160 time result, not a true two-spatial-dimensional field.

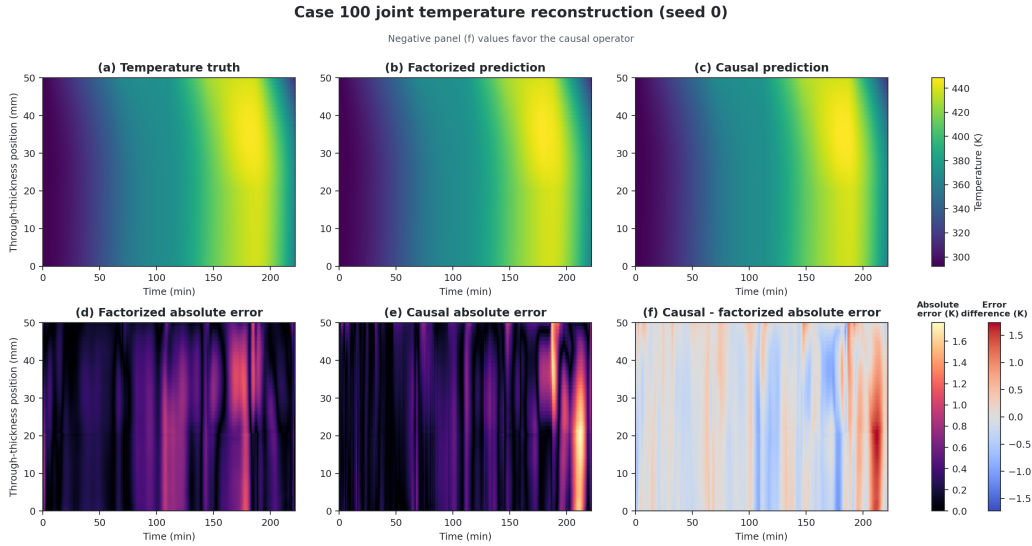


Figure 2: Representative joint 1+1-D temperature reconstruction for fixed test case 100 and seed 0. The factorized and causal models were selected by the validation objective. This single-case display is descriptive and does not establish statistical superiority.

161 The factorized and causal joint models used 198,282 and 225,162 pa-
 162 rameters, respectively, compared with 15,076,467 parameters across 51 in-
 163 dependently trained legacy models. Their seed-0 complete-case temperature
 164 relative L_2 means were 0.001192 and 0.001310; all saved cure predictions re-
 165 spected bounds and monotonicity, and the causal model showed exactly zero
 166 prefix change in its fixed future-perturbation test. These are staged, one-
 167 seed validation results and are included only to demonstrate the software’s
 168 joint-field and causality workflows.

Table 3: Evidence used to validate the current software release. Performance entries marked descriptive or validation-only are not confirmatory claims.

Stage	Population	Verified result	Claim boundary
P0–P1	Public Case1; 200 cycles, 51 positions, 223 times	Nine legacy defects regression-tested; exact and corrected field workflows independently reconstructed	Reproduction and audit behavior; no corrected-accuracy gain claimed
P2	125 complete test cases; seed 0	Joint factorized rel $L_2 = 0.001192$; causal rel $L_2 = 0.001310$; zero causal prefix change	Descriptive one-seed phase gate; 1+1-D only
P3	200 public solver cases	Mean temperature rel $L_2 = 0.001143$; maximum global energy residual 5.304×10^{-13}	Open-solver agreement; proprietary COM-SOL parity not established
P4	512 generated true-2-D cases	27 solver checks passed; zero failed cases; deterministic replay passed	Public-data-anchored numerical benchmark; no geometry-OOD claim
P5	Label-free restriction tests and one-seed validation pilot	Causal restriction and future-invariance gates passed; transfer pilot passed at budgets 8 and 16	Initialization and functionality evidence; pilot comparison is validation-only

169 4. Impact

170 CD-CureNO’s primary present impact is methodological infrastructure.
171 First, it makes the behavior of a legacy curing-operator codebase inspectable.
172 Exact and corrected paths remain separately named, allowing a researcher
173 to reproduce reported behavior without quietly repairing it, then compare
174 against a fairer implementation. The audit detected and regression-tested
175 nine issues, including normalization leakage, an unused smoothness term,
176 inconsistent function calls, and incomplete field output expectations. This

177 reduces the risk that later scientific conclusions rest on undocumented im-
178 plementation differences.

179 Second, the software connects conservative simulation to operator learn-
180 ing in one repository. The one-dimensional solver is checked against 200
181 public cases, refinement, and energy closure. The two-dimensional solver is
182 checked against an extruded one-dimensional limit, a method-of-lines BDF
183 calculation, energy invariants, deterministic replay, and complete dataset
184 manifests. The same case identifiers, masks, coordinates, and hashes then
185 flow into training and evaluation. This shared contract is useful for studying
186 transfer without treating generated labels as unquestioned ground truth.

187 Third, CD-CureNO turns restriction preservation and causal response
188 into executable tests. A researcher can inspect which tensors were copied,
189 which were initialized, whether a lateral zero mode can change the source
190 mapping, and whether any future input changes a predicted prefix. These
191 facilities can support new questions about when lower-dimensional pretrain-
192 ing helps, when the benefit disappears as target labels increase, and whether
193 physical invariants improve worst-case behavior. They can also be reused
194 as test patterns for other evolution equations with a known dimensional-
195 reduction limit.

196 Fourth, the managed-run and reporting layers preserve negative evidence.
197 Interrupted sessions, failed development variants, superseded cohorts, and
198 unknown wall-time intervals are retained rather than zero-imputed or re-
199 placed. Manuscript tokens can be resolved only from a verified statistics
200 pointer and an explicit run population. This supports the software-citation
201 principle that a cited version should be identifiable and recoverable [9].

202 The current repository has no defensible public download, user, or com-
203 mercial adoption count, so this article makes no adoption claim. At sub-
204 mission, the tagged GitHub release, license, README, and developer doc-
205 umentation named in Table 1 must be public. CD-CureNO is released un-
206 der the OSI-approved Apache License 2.0, which permits use, modification,
207 and redistribution, including commercial use, subject to its terms. Exter-
208 nal/ResFNO code and data must not be redistributed under an assumed
209 license; the release should retain only the submodule pointer or documented
210 acquisition/checksum workflow allowed by the upstream terms.

211 Scientific scope also remains bounded. The canonical benchmark uses
212 a rectangular grid, reference-temperature conductivity, and a unidirectional
213 material model. Geometry OOD is deferred. The BDF cross-check is in-
214 dependent in flux assembly and integration but shares physical inputs. P2

215 and P5 model comparisons are one-seed or validation-only. The unfinished
216 P6 confirmatory campaign and context-only external sources are not used in
217 this paper as evidence of superiority, broad generalization, or experimental
218 validation.

219 5. Conclusions

220 CD-CureNO packages the complete research workflow needed to study
221 cross-dimensional neural operators for composite curing: legacy audit, con-
222 servative one- and two-dimensional solvers, immutable data manifests, joint
223 and causal models, deterministic restriction-preserving inflation, resumable
224 managed execution, independent evaluation, and traceable reporting. The
225 current validation demonstrates software correctness and staged scientific
226 readiness while explicitly preserving the boundary between validation and
227 confirmation. The package is intended to make future claims easier to audit,
228 reproduce, narrow, or reject when the evidence requires it.

229 Data and code availability

230 The exact software version described in this article is available from the
231 following repository:

232 <https://github.com/Jinhong-Yang/CD-CureNO/tree/v0.0.1>

233 The open-source release license is:

234 Apache License 2.0 (SPDX: Apache-2.0)

235 Generated arrays, public-data acquisition steps, manifests, and redis-
236 tribution restrictions are documented in the release README. Upstream
237 ResFNO artifacts remain subject to their own license status and are not
238 relicensed by CD-CureNO.

239 CRediT author statement

240 **[SOFTWAREX-REPLACE: CRediT author contribution state-**
241 **ment]**

242 Funding

243 This work was supported by the Institute of Information & Communi-
244 cations Technology Planning & Evaluation (IITP) – Innovative Human Re-
245 source Development for Local Intellectualization program grant funded by
246 the Korea government (MSIT) (IITP-2026-RS-2024-00436773).

247 Acknowledgements

248 None.

249 Declaration of competing interest

250 The authors declare that they have no known competing financial inter-
251 ests or personal relationships that could have appeared to influence the work
252 reported in this paper.

253 References

- 254 [1] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stu-
255 art, A. Anandkumar, Fourier neural operator for parametric partial dif-
256 ferential equations, in: International Conference on Learning Represen-
257 tations, 2021.
258 URL <https://openreview.net/forum?id=c8P9NQVtmn0>
- 259 [2] S. Amini Niaki, E. Haghighat, T. Campbell, A. Poursartip, R. Vaziri,
260 Physics-informed neural network for modelling the thermochemical cur-
261 ing process of composite-tool systems during manufacture, Computer
262 Methods in Applied Mechanics and Engineering 384 (2021) 113959.
263 doi:10.1016/j.cma.2021.113959.
- 264 [3] G. Chen, Y. Li, X. Liu, C. Mehdi-Souzani, Q. Meng, J. Zhou, X. Hao,
265 Physics-guided neural operator for data-driven composites manufacturing
266 process modelling, Journal of Manufacturing Systems 70 (2023) 217–229.
267 doi:10.1016/j.jmsy.2023.07.010.
- 268 [4] Q. Meng, Y. Li, X. Liu, G. Chen, X. Hao, A novel physics-informed neu-
269 ral operator for thermochemical curing analysis of carbon-fibre-reinforced
270 thermosetting composites, Composite Structures 321 (2023) 117197. doi:
271 10.1016/j.compstruct.2023.117197.
- 272 [5] G. Chen, Y. Li, X. Liu, Q. Meng, J. Zhou, X. Hao, Residual fourier
273 neural operator for thermochemical curing of composites, arXiv preprint
274 arXiv:2111.10262 (2021). doi:10.48550/arXiv.2111.10262.
- 275 [6] A. Hemmasian, A. Barati Farimani, Pretraining a neural operator in lower
276 dimensions, Transactions on Machine Learning Research (2024).
277 URL <https://openreview.net/forum?id=ZewaRoZehI>

- 278 [7] A. Kashi, A. Daw, M. Gopalakrishnan Meena, H. Lu, Learning the
279 boundary-to-domain mapping using lifting product fourier neural oper-
280 ators for partial differential equations, in: ICML 2024 AI for Science
281 Workshop, 2024. [arXiv:2406.16740](https://arxiv.org/abs/2406.16740).
282 URL <https://arxiv.org/abs/2406.16740>
- 283 [8] L. Liu, K. Nath, W. Cai, A causality-deeponet for causal responses of lin-
284 ear dynamical systems, Communications in Computational Physics 35 (5)
285 (2024) 1194–1228. doi:10.4208/CICP.0A-2023-0078.
- 286 [9] A. M. Smith, D. S. Katz, K. E. Niemeyer, FORCE11 Software Citation
287 Working Group, Software citation principles, PeerJ Computer Science 2
288 (2016) e86. doi:10.7717/peerj-cs.86.